

《计算机网络》课程设计报告

2026 年 1 月 9 日

目录

第一章 概述	3
1.1 设计目标	3
1.2 主要流程	3
1.3 主要功能	3
1.4 开发环境	4
1.5 参考资料	4
第二章 模块划分及软件流程图	5
2.1 args_handler 参数处理	5
2.2 dns_parser 报文解析	6
2.2.1 请求报文解析	6
2.2.2 答复报文构造	8
2.3 file_reader 文件读取	11
2.3.1 将文件载入内存	11
2.3.2 从内存找域名	12
2.4 mapping 序号映射	13
2.5 network 网络部分	15
2.5.1 初始化	16
2.5.2 主循环	16
2.6 主函数入口	17
2.7 函数调用图及软件流程图	18
第三章 测试	20

目录	2
第四章 问题及解决	21
4.1 C 语言位域问题	21
4.2 bind 失败	21
4.3 AAAA 记录查询	22
第五章 总结和心得体会	23

第一章 概述

1.1 设计目标

本课程设计旨在完成一个 DNS 中继程序，实现不良网站拦截功能、服务器功能、中继功能。

1.2 主要流程

本设计的执行流程简述如下：

程序启动时，解析命令行参数，加载本地 DNS 记录文件，初始化网络部分（包括 UDP 套接字和 DNS 服务器地址），在主循环中接收来自客户端的 DNS 请求，若在本地记录中找到请求中对应的 IP 地址，则构造响应报文回答给客户端，否则向外部 DNS 服务器转发请求。若对应的 IP 地址为 0.0.0.0，发送一个带有拒绝信息的 DNS 响应报文给客户端以实现屏蔽功能。

1.3 主要功能

本课程设计完成了以下功能：

1. 参数解析
2. 不良网站拦截功能、服务器功能、中继功能
3. 调试信息输出
4. 支持并发查询
5. 支持序号转换

1.4 开发环境

开发环境如下：

- **操作系统**：Windows 11
- **代码编辑器**：Visual Studio Code
- **中间层**：Cygwin
- **构建工具**：CMake 3.28.1
- **编译器**：gcc 11.4.0

由于使用了 GNU argp 库，故使用 Cygwin 提供类 Unix/Linux 环境。

1.5 参考资料

本课程设计完全由我们自行实现，主要的参考资料为RFC 1035。

第二章 模块划分及软件流程图

我们的设计主要有以下模块：

- `args_handler` 参数处理
- `dns_parser` 报文解析
- `file_reader` 文件读取
- `mapping` 序号映射
- `network` 网络部分
- 主函数入口

2.1 `args_handler` 参数处理

含文件 `args_handler.h` `args_handler.c`。

使用 GNU `argp` 库，实现了命令行参数的解析和初始化，支持设置调试等级、DNS 服务器地址和本地文件地址，并在初始化时打印相关信息。

支持的命令行参数如下：

- `-d`：输出简略的调试信息。
- `-dd`：输出详细的调试信息。

- `<dns-server-ipaddr>`: 形如 a.b.c.d 的地址, 用于初始化 DNS 服务器地址。有 $0 \leq a, b, c, d \leq 255$ 。
- `<filename>`: 本地文件名, 用于初始化读入的 ip-域名表的位置。

一个示例:

```
1 .\dnsrelay -dd 114.114.114.114 /home/user/localdns.txt
```

模块提供的接口如下:

```
1 void args_init(int argc, char *argv[]);
```

参数:

- `argc`: 参数个数
- `argv`: 参数列表

2.2 dns_parser 报文解析

含文件 `dns_parser.h` `dns_parser.c`。

本模块实现对 DNS 报文的处理和构造, 主要有解析报文的各字段、构造答复报文。

关于 dns 报文的具体格式, 在 RFC 1035 中已经描述的很清楚了, 这里不再赘述。

2.2.1 请求报文解析

解析 `name` 字段

根据 RFC 1035 的定义, `name` 字段由“长度”和“标签”组成, “标签”即由点号分隔的域名的一部分, 在“标签”前加上其长度, 即由“长度”代替点号, 以长度为 0 标识结束。例如“example.com”

被表示为“7example3com0”，注意此处的数字不是 ascii 字符的数字，而是数字值。7 即 0x07。

接口：

```
1 char* name_parse(char *buf);
```

参数：

- buf：待解析的缓冲区，是一个 Question 字段。

解析请求类型

在 DNS 请求报文中，Question 字段包含了查询的类型。根据 RFC 1035 的定义，查询类型字段位于 Question 字段的末尾，紧跟在域名字段之后。解析请求类型的函数需要遍历域名字段，找到查询类型字段的位置，并根据其值确定查询类型。

宏定义：

```
1 #define TYPE uint16_t
2 #define A 1
3 #define NS 2
4 #define MD 3
5 #define MF 4
6 #define CNAME 5
7 #define SOA 6
8 #define MB 7
9 #define MG 8
10 #define MR 9
11 #define NULL_R 10
12 #define WKS 11
13 #define PTR 12
14 #define HINFO 13
15 #define MINFO 14
16 #define MX 15
17 #define TXT 16
18 #define AAAA 28
```

接口：

```
1 TYPE get_question_type(const unsigned char *question);
```

参数：

- question：待解析的 Question 字段。

返回值：

- 返回解析出的查询类型。

解析过程：首先，函数遍历 Question 字段中的域名部分，直到遇到长度为 0 的标签，标识域名部分结束。接着，函数读取紧跟在域名部分之后的两个字节，这两个字节表示查询类型。最后，函数通过 switch 语句，根据查询类型的值，输出相应的调试信息，并返回查询类型。

2.2.2 答复报文构造

在 DNS 中，当服务器接收到客户端的请求后，需要构造相应的答复报文。答复报文主要包括三个部分：头部、资源记录（RR）和回复报文。下面详细介绍这三个部分的构造过程。

头部构造

头部包含了请求报文的基本信息和服务器的应答信息。构造头部时，首先清空待填充的报文头。接着，将请求报文头中的 ID 和其他必要字段复制到应答报文头中，并设置 qr 字段为 1，表示这是一个响应报文。根据应答类型（如接受或拒绝），设置 rcode 字段。如果是拒绝应答，rcode 设置为 5；如果是接受应答，rcode 设置为 0，并且需要设置 qdcount 和 ancounat 字段。最后，设置其他固定字段，如 aa、tc、rd、ra 和 z 等。

接口：

```
1 void fill_header(Header *to_fill, const Header *src,
    const int type);
```

参数:

- to_fill: 待填充的报文头。
- src: 请求报文头。
- type: 类型, REFUSE 表示拒绝, ACCEPT 表示接受。

资源记录 (RR) 构造

资源记录 (RR) 包含了查询结果的具体信息。构造资源记录时, 首先需要初始化一个 RR 结构体, 设置其类型为 A 记录, 类为 IN 表示因特网, TTL 为 60 秒 (这是可变的), rdata 长度为 4 字节, 即 4 字节的 ipv4 地址。

在 DNS 报文中, 名称字段可以使用指针来减少报文的长度。根据 RFC 1035 的定义, 指针的格式为两个字节, 其中前两位为“11”, 表示这是一个指针, 后面的 14 位表示指针所指向的位置的偏移量。

设置指针 0xc00c 指向 name 字段, 表示资源记录的名称部分。偏移量的值为 12 (十进制), 表示指针指向报文的第 12 个字节处 (头长度是 12 字节)。然后, 将 RR 结构体的内容复制到 RR 缓冲区中, 跳过 name 字段。最后, 将查询结果 (4 字节的 ipv4 地址) 复制到 RR 缓冲区的末尾。

接口:

```
1 void construct_RR(unsigned char *rr, const uint32_t
    result);
```

参数:

- rr: 待填充的 RR。

- `result`: 查询结果 (ipv4 地址)。

回复报文构造

回复报文包含了头部、请求报文的部分内容和资源记录。构造回复报文时，首先将响应报文头复制到回复报文缓冲区的起始位置。接着，将请求报文的部分内容（除头部外）复制到回复报文缓冲区的相应位置，这部分内容通常包括请求的域名和查询类型。之所以这么做是因为我们观察到使用运营商的 DNS 服务器时也会在回复中复制一份 Question 字段。这样做可能的好处有：1. 方便；2. 便于资源记录部分使用指针指向 name。最后，将资源记录复制到回复报文缓冲区的末尾，资源记录包含了查询结果的具体信息，如 IP 地址等。

接口：

```
1 void construct_response(unsigned char *response, Header
    *response_head, const unsigned char *buf, const int
    len, const unsigned char *rr);
```

参数：

- `response`: 待填充的响应报文。
- `response_head`: 响应报文头。
- `buf`: 请求报文。
- `len`: 请求报文长度。
- `rr`: 资源记录。

通过以上步骤，服务器可以构造出完整的 DNS 答复报文，并将其发送给客户端，从而完成一次 DNS 查询的应答过程。

2.3 file_reader 文件读取

含文件 `file_reader.h` `file_reader.c`。

在程序中，需要将本地 DNS 记录文件载入内存，并在内存中查找域名对应的地址。下面详细介绍这两个过程。

2.3.1 将文件载入内存

将文件载入内存的过程如下：

- 打开文件：使用 `fopen` 函数打开本地 DNS 记录文件。如果文件打开失败，输出错误信息并退出程序。
- 初始化缓冲区：定义两个缓冲区 `addr_string` 和 `name`，分别用于存储地址和域名，并将它们初始化为空。
- 读取文件内容：使用 `fscanf` 函数逐行读取文件内容，每行包含一个地址和一个域名。将读取到的地址转换为整数形式，并存储在 `pairs` 数组中。如果地址无效，输出错误信息。
- 存储域名：将读取到的域名存储在 `pairs` 数组中，并为其分配内存。
- 清空缓冲区：每次读取完一行后，清空缓冲区，以便读取下一行。
- 关闭文件：使用 `fclose` 函数关闭文件。如果文件关闭失败，输出错误信息并退出程序。
- 排序：使用 `qsort` 函数对 `pairs` 数组进行排序，以便后续查找。

接口：

```
1 void load_file();
```

2.3.2 从内存找域名

从内存中查找域名对应地址的过程如下：

- 初始化查找项：定义一个 `pair` 结构体 `p`，并将待查找的域名赋值给 `p.name`。
- 查找：使用 `bsearch` 函数在 `pairs` 数组中查找 `p`。`bsearch` 函数使用二分查找算法，能够快速找到目标项。
- 返回结果：如果找到匹配的项，返回其对应的地址；如果未找到，返回 `UINT32_MAX`。

接口：

```
1 uint32_t find_entry(char *name);
```

参数：

- `name`：待查找的域名。

返回值：

- 返回查找到的地址，如果未找到，返回 `UINT32_MAX`。

qsort 和 bsearch 的使用

在文件处理过程中，使用了 `qsort` 和 `bsearch` 函数来提高查找效率。

- `qsort`：用于对 `pairs` 数组进行排序。排序的依据是域名的字典序。排序后，可以使用二分查找算法快速查找目标项。
- `bsearch`：用于在 `pairs` 数组中查找目标项。`bsearch` 函数使用二分查找算法，能够在对数时间内找到目标项。

比较函数：

```
1 static int cmp(const void *a, const void *b);
```

参数:

- a: pair 指针。
- b: 另一个 pair 指针。

返回值:

- 返回比较结果。

通过以上步骤, 可以将本地 DNS 记录文件载入内存, 并在内存中快速查找域名对应的地址, 从而提高 DNS 中继程序的效率。

2.4 mapping 序号映射

含文件 mapping.h mapping.c。

在 DNS 中继程序中, 需要将请求报文的 ID 与客户端地址进行映射, 以便在接收到 DNS 服务器的响应时能够正确地将响应报文返回给客户端。为了实现这一功能, 我们使用了直接映射, 并保证操作的时间复杂度为 $O(1)$ 。下面详细介绍这一过程。

在 mapping.h 文件中, 定义了用于映射 ID 的结构体 Map, 以及相关的宏和函数声明。

```
1 #define bool uint8_t
2 #define false 0
3 #define true 1
4 #define deleted 2
5
6 typedef struct {
7     uint16_t id;
8     struct sockaddr_in addr;
9     bool is_arrived;
10 } Map;
```

其中 `bool` 用于表示 `bool` 类型, 但是增加了 `deleted` 状态。Map 结构体用于映射 `id`, 含有三个成员: `id` 表示原 `id`, `addr` 表示映射前客户端的地址, `is_arrived` 表示是否收到 `dns` 服务器的回复。

具体实现如下: 将原始请求 `id` 映射一个计数器值 `cnt`, 将该计数器值作为新的 `id` 替换原始 `id`, 再将替换后的报文发送给 `dns` 服务器。同时将 `cnt` 作为下标, 将客户端地址和原始 `id` 和 `is_arrived` 标志作为值存入 `map` 数组中。

收到 `dns` 服务器的回复后, 取出 `id`, 从 `map` 数组中找到映射前的 `id` 和地址, 把回复发给客户端, 同时置 `is_arrived` 为真。

这种方法适合于并发查询, 因为每个查询都可以在常数时间内完成映射和查找操作, 从而提高了系统的并发处理能力。

接口:

```
1 uint16_t convert_id_to_cnt(Header *head, struct
    sockaddr_in cli_addr);
```

作用: 改写头的 `id` 字段, 记录 `map`。

参数:

- `head`: 消息头。
- `cli_addr`: 客户端地址。

返回值:

- 返回映射后的 `ID`。

```
1 bool convert_cnt_to_id(Header *head, struct sockaddr_in
    *local_cli_addr);
```

作用: 改写头的 `id` 字段, 给 `local_cli_addr` 赋原来的值。

参数:

- `head`: 消息头。
- `local_cli_addr`: 客户端地址。

返回值:

- 返回是否成功, 超时未收到 DNS 服务器的回复返回 `false`。

超时管理

在用来序映射序号的 Map 结构体中, 我们还留了一个 `is_arrived` 标志位, 用来进行超时管理。其工作流程如下:

当程序将收到的请求转发给 DNS 服务器时, 调用 `sleep` 函数睡眠一个常数时间, 同时置 `is_arrived` 为 `false`。当收到服务器的回复之后, 将 `is_arrived` 置为 `true`, 表示到达。

当睡眠时间结束后, 如果 `is_arrived` 仍然是 `false`, 表示规定的时间内没有收到回复, 可以认为发生了超时。所以将 `is_arrived` 置为 `deleted`, 这样一来, 以后再次收到响应报文时, 如果 `is_arrived` 是 `deleted` 则将其丢弃。

通过以上步骤, 实现了服务器响应超时的管理。需要注意的是, 由于使用了多个线程, 所以睡眠不会阻塞程序的正常运行。

2.5 network 网络部分

含文件 `network.h` `network.c`。

实现了 DNS 中继服务的网络通信功能, 包括 UDP 套接字的创建、DNS 服务器地址的初始化、消息的接收与分类处理, 以及主处理循环。

模块初始化时, 会创建一个 UDP 套接字用于接收客户端的 DNS 查询请求, 并将请求转发给配置的 DNS 服务器。同时, 监听 DNS 服务器的响应, 并将响应转发回客户端。

支持的功能如下:

- 创建并配置 UDP 套接字, 包括地址复用。
- 初始化 DNS 服务器地址。

- 接收客户端的 DNS 查询请求，并根据本地 DNS 表或转发到 DNS 服务器进行处理。
- 接收 DNS 服务器的响应，并将响应转发给原始请求的客户端。
- 通过线程分离处理每个接收到的消息，提高并发处理能力。

2.5.1 初始化

接口：

```
1 void network__init(void);
```

功能：初始化网络模块，包括创建 UDP 套接字和设置 DNS 服务器地址。

首先创建套接字和地址结构体，设置地址复用，然后绑定套接字和地址，最后初始化 DNS 服务器地址。有关地址复用，会在下文详细介绍。

2.5.2 主循环

接口

```
1 void main__process(void);
```

功能：运行 DNS 中继服务的主处理循环，接收并处理 DNS 查询请求。

在主循环中，接受来自客户端或 DNS 服务器的消息，为每个消息创建一个线程处理，根据消息类型调用相应的处理函数。首先，将消息交给 `message_classify` 函数进行分类，该函数如下：

```
1 static void *message__classify(void *arg);
```

功能：对接收到的消息进行分类，并根据类型调用相应的处理函数。

`message_classify` 函数根据消息的类型进行不同的处理，如果是客户端的查询请求，调用 `handle_message` 函数处理。否则为 DNS 服务器的响应，进行 ID 转换，找到对应的客户端地址，将响应转发给客户端。

`handle_message` 函数如下：

```
1 static void handle_message(unsigned char *buf, const int  
    len, struct sockaddr_in cli_addr, socklen_t  
    cli_addr_len);
```

功能：处理来自客户端的 DNS 查询请求。

首先从报文中解析出域名，然后在本地 DNS 表中查找对应的 IP 地址，如果找到，构造响应报文（具体地址或屏蔽）并发送给客户端。如果未找到，将请求转发给 DNS 服务器，等待 DNS 服务器的响应。

需要注意的是，我们还考虑了可能的超时情况，如果超时未收到 DNS 服务器的响应，后续再收到响应时会丢弃。

以及，如果表中找到了域名，但是请求类型为 AAAA，我们会向外部 DNS 服务器进行查询。

本模块使用多线程处理接收到的每个 DNS 查询请求，通过分离线程提高响应速度和并发处理能力。本模块是 DNS 中继服务的核心，负责所有网络通信和消息转发任务。

2.6 主函数入口

含文件 `dnsrelay.c`。

包含 `main` 函数，调用各个模块的初始化函数，然后进入主循环。具体调用如下：

```
1 int main(int argc, char *argv[]) {  
2     args_init(argc, argv); // 参数解析  
3     load_file(); // 文件读取  
4     network_init(); // 网络初始化  
5     main_process(); // 主循环  
6     return 0;  
7 }
```

2.7 函数调用图及软件流程图

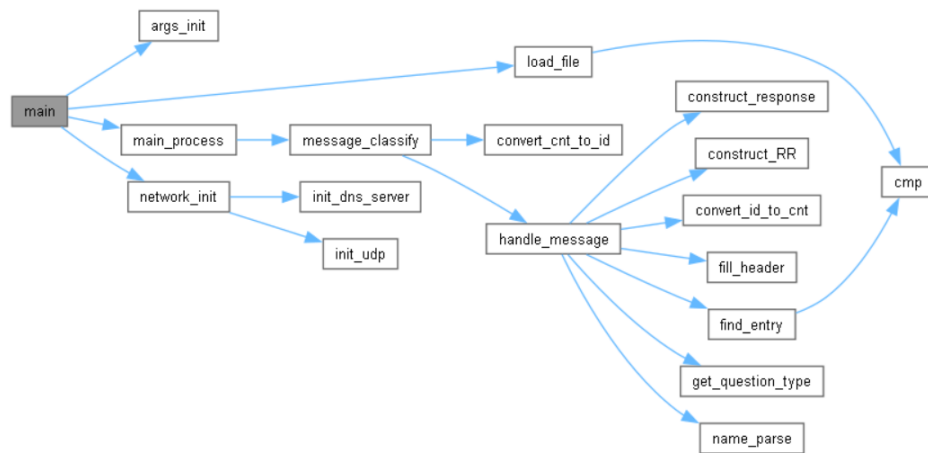


图 2.1: 函数调用图

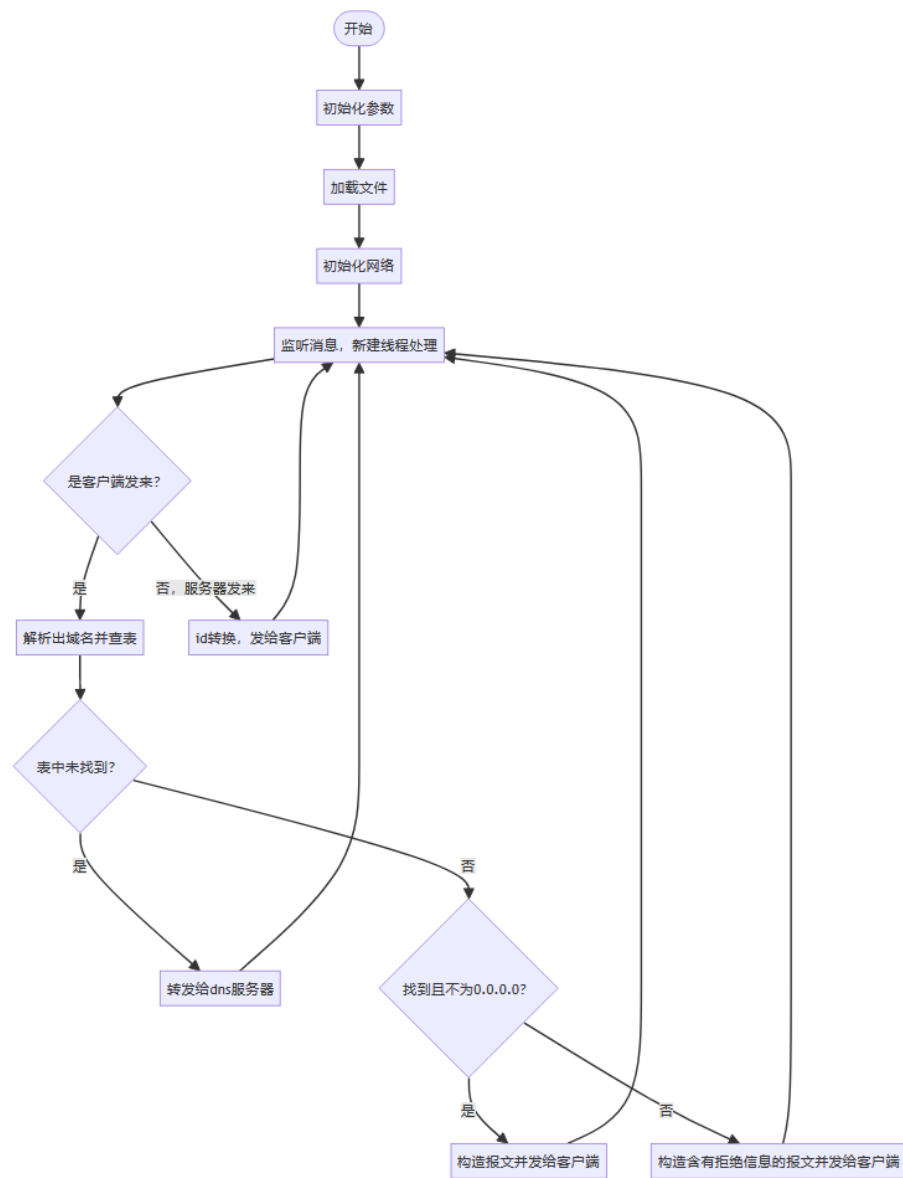


图 2.2: 软件流程图

第三章 测试

略。

第四章 问题及解决

实现过程中，主要遇到了三个问题，下面分别阐述。

4.1 C 语言位域问题

在对收到的报文进行处理时，我们发现，从客户端收到的查询报文中的 qr 字段为 1。按理来说查询报文中的此字段应该为 0，响应报文应该为 1，这让我们百思不得其解。我们将接收到的字节流打印出来，发现 qr 字段的确为 0，于是推测可能是 header 字段的数据结构定义出错。通过实验验证，在小端机器中，定义位置靠前的位域是低位比特，而在网络中，位置靠前的为高位比特。因此在一个字节内部，应该用与 RFC 1035 相反的顺序来定义标志位。

4.2 bind 失败

在将套接字和地址绑定时，我们发现一直失败。通过 `perror` 打印出的错误信息为 `Address already in use`。

使用 `netstat -ano` 命令查看当前连接，发现一项 Windows 的系统服务占用了 udp 的 53 端口。经查询，使用 `setsockopt` 函数，将地址设置为可以复用，此问题得到解决。

4.3 AAAA 记录查询

当客户端查询的域名在本地文件中有相应的匹配时，如果查询的是 A 记录，则会直接返回；如果查询的是 AAAA 记录，也会返回一个 IPV4 的地址。

针对这种情况，我们做了修改，如果本地文件中有对应的域名，但是查询是 AAAA 记录，则将 AAAA 记录的查询中继出去。

第五章 总结和心得体会

通过设计并实现 DNS 中继服务器，我们获得了以下扩展的经验和技能：

- **深入理解 DNS 协议和 RFC 标准：**在阅读 RFC1034 和 RFC1035 协议过程中，掌握了 DNS 包结构、功能实现原理以及实际应用的详细知识。熟练掌握了英文协议文档的阅读和分析技能，能够理解协议规范并将其应用于实际的软件设计和开发中。
- **通信协议设计和分析能力：**通过使用 Wireshark 等抓包工具，熟练掌握了数据包的捕获、测试和分析技能，能够深入分析通信过程中的数据交换和协议实现细节。基于包结构分析，能够系统性地设计和优化通信协议，确保协议的高效性和可靠性。
- **Socket 编程和网络通信实现：**掌握了 Socket 编程的核心知识要点，能够熟练使用 Socket API 实现主机和服务端之间的通信。在实现 DNS 中继服务器的过程中，应用了 Socket 编程接口完成了 UDP 通信机制的实现，从而实现了服务器的软件功能。
- **自主学习和问题解决能力：**通过资料学习和与他人的沟通交流，积极探索和解决实际问题，提升了在计算机网络领域自主学习的能力。在实践中培养了解决复杂问题的能力，包括理解和应用新技术、调试和优化系统性能等方面。

- **理论知识与实际应用的结合：**将在计算机网络课程中学习的理论知识应用于实际项目设计中，深化了对网络通信原理和协议的理解。实际项目的开发过程中，能够将学术概念转化为实际可执行的系统设计，从而提升了对计算机网络与通信系统设计和开发能力的理解和实践。

提交内容说明

提交的内容包括：

- 实验报告
- .c 源文件：src 文件夹
- .h 头文件：include 文件夹
- 二进制文件：build 文件夹
- CMakeLists.txt 文件